

Cloud Image Registry & Distribution Laboratory Report

Laboratory Report: Cloud Image Registry & Distribution

I. Experimental Purpose

1. Practice version control using Docker Tags to manage different application versions.
2. Validate decoupled deployment by testing disaster recovery from cloud registry without source code.
3. Implement security best practices using Personal Access Tokens instead of plaintext passwords.

II. Experimental Procedures

Step 1: Secure Authentication with PAT

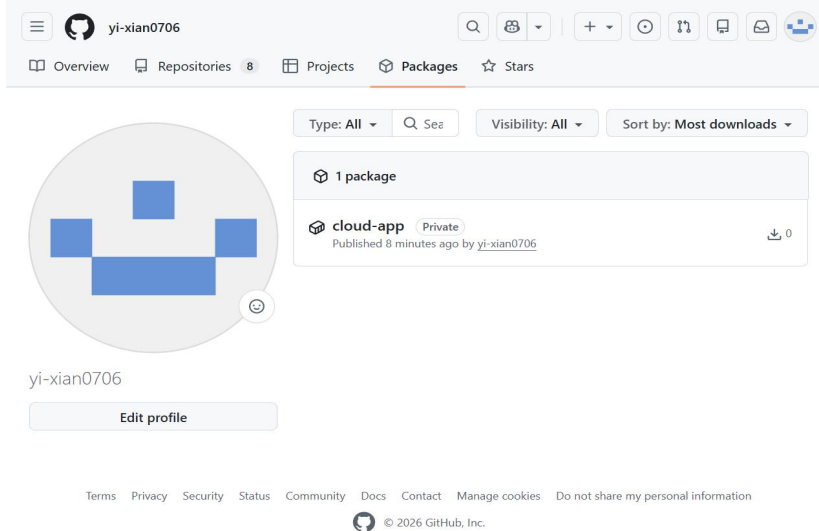
First, I generated a Personal Access Token (PAT) in GitHub Developer Settings with write:packages permission. Then I logged into GHCR using the token in terminal.

```
• @yi-xian0706 → /workspaces/Cloud-Computing (main) $ export  
  CR_PAT=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  
• @yi-xian0706 → /workspaces/Cloud-Computing (main) $ echo $C  
  R_PAT | docker login ghcr.io -u yixian-0706 --password-stdi  
  n  
  
WARNING! Your credentials are stored unencrypted in '/home/  
codespace/.docker/config.json'.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/go/credential-store/  
  
Login Succeeded  
○ @yi-xian0706 → /workspaces/Cloud-Computing (main) $ █
```

Step 2: Tag and Push v1.0 Image

I built the local Docker image and tagged it in GHCR format: ghcr.io/yi-xian0706/cloud-app:v1.0, then pushed it to GitHub Container Registry.

```
@yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker
push ghcr.io/yi-xian0706/cloud-app:v1.0
The push refers to repository [ghcr.io/yi-xian0706/cloud-ap
p]
f95ac1db5b48: Pushed
80a93e71c899: Pushed
abaae85d1626: Pushed
43f834d60d8a: Pushed
fbaed3f7fcbe: Pushed
a5008f4a4b25: Pushed
de1b677d8c00: Pushed
6a0ac1617861: Pushed
94d083cf706a: Pushed
e654dbbbb9e1: Pushed
v1.0: digest: sha256:ebea4e5700327bbc1d2223f4d02e26e25fb707
5660720fb363ea7c15f000530f size: 856
```



Step 3: Version Upgrade to v2.0 (Layered Storage)

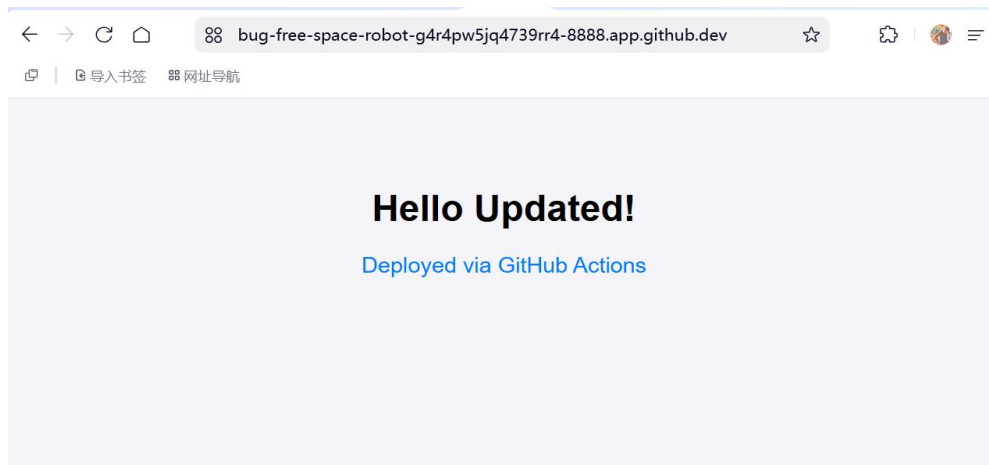
After modifying index.html for v2.0, I rebuilt and pushed the new version. I observed that most layers showed "Layer already exists" because Docker only uploads the changed layer.

```
@yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker
push ghcr.io/yi-xian0706/cloud-app:v2.0
eeb200d3ff71: Pushed
7c4f1289bb22: Pushed
94d083cf706a: Layer already exists
abaae85d1626: Layer already exists
43f834d60d8a: Layer already exists
de1b677d8c00: Layer already exists
6a0ac1617861: Layer already exists
a5008f4a4b25: Layer already exists
e654dbbbb9e1: Layer already exists
fbaed3f7fcbe: Layer already exists
v2.0: digest: sha256:fdcdf9e977cbf81fe9226a6fbff02837df25d4b
55645c2947909399b301f3d885 size: 856
@yi-xian0706 → /workspaces/Cloud-Computing (main) $
```

Step 4: Disaster Recovery Test

I deleted all local images using "docker system prune -a", then pulled and ran v1.0

directly from GHCR without any source code files. This proved the application is independent from local environment.



When running v2.0 on the same port, I encountered "port is already allocated" error. I solved it by stopping the old container and using a different port.



Step 5: Alpine Image Optimization

I compared standard nginx image with nginx:alpine. The results showed nginx:latest was 241MB while nginx:alpine was only 93.6MB.

```
@yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker images | grep nginx
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx:alpine          8b1e78743a03          93.6MB
27MB
nginx:latest          5aca99593157          241MB
66MB
@yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker images | grep cloud-app
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-app:alpine      1449178ac160          113MB
35.2MB
cloud-app:standard    2d8a41235937          92.7MB
26MB U
ghcr.io/yi-xian0706/cloud-app:v1.0 ebea4e570032          92.7MB
26MB U
ghcr.io/yi-xian0706/cloud-app:v2.0 386508af92c9          92.7MB
26MB U
```

Step 6: Git Version Control

Finally, I saved all changes using git add, commit, and push commands.

```
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ git add .
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ git commit -m "Finished Lab 03 - Cloud Image Registry"
[main 74ea504] Finished Lab 03 - Cloud Image Registry
 2 files changed, 7 insertions(+), 5 deletions(-)
 create mode 100644 Dockerfile.alpine
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 611 bytes | 611.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/yi-xian0706/Cloud-Computing
 78b25e3..74ea504  main -> main
○ @yi-xian0706 → /workspaces/Cloud-Computing (main) $ █
```

III. Reflection Questions

1. Why was pushing v2.0 much faster than v1.0?

Pushing v2.0 was faster because of Docker's layer caching mechanism. Only the modified HTML layer needed to be uploaded, while all other base layers were reused from v1.0, showing "Layer already exists". This layered storage saves time and bandwidth.

2. Why is PAT more secure than password?

PAT is more secure because it allows fine-grained permission control (only packages access), can be revoked anytime without affecting the main password, supports expiration dates, and limits damage scope even if leaked.

3. Why is Alpine the industry standard for microservices?

Alpine Linux is the industry standard because it has much smaller image size (saves storage and network), faster cold start time, smaller attack surface for better security, and reduces overall cloud costs.